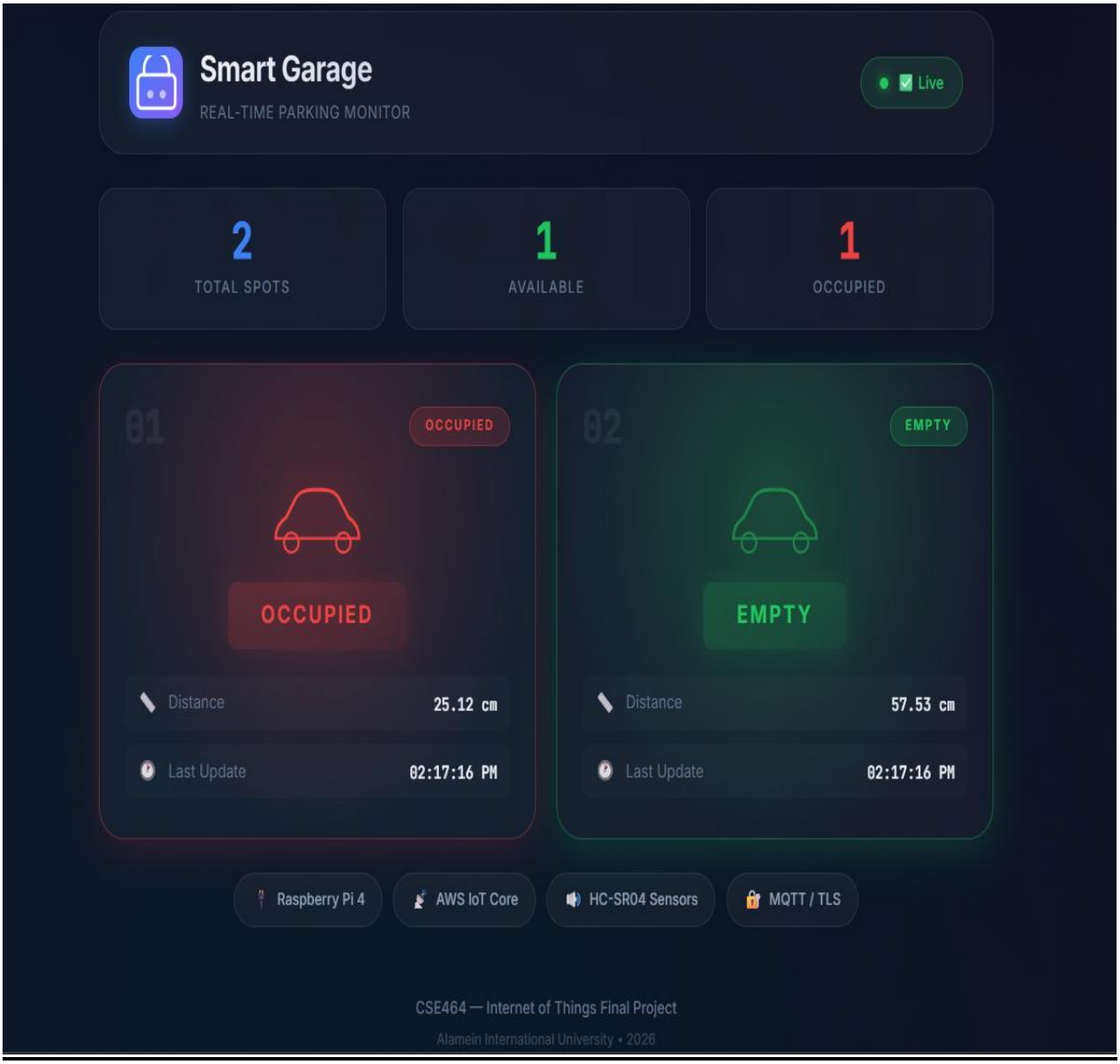
Alamein International University
Department of Computer Science & Engineering
Spring 2025

Final Project Report

Smart Garage Monitor

A Smart Parking Solution for Al-Alamein Smart City



1. Project Overview & Problem Definition

1.1 Problem Statement

Inadequate parking management is one of the most pressing urban challenges facing rapidly growing smart cities such as Al-Alamein. Drivers frequently circulate for extended periods searching for available parking spaces, resulting in increased fuel consumption, vehicle emissions, traffic congestion, and significant time wastage. In the absence of a real-time monitoring system, there is no reliable mechanism for drivers to determine the availability of parking spots without physically approaching them.

The Al-Alamein Smart City initiative aims to integrate advanced IoT infrastructure across multiple urban domains, including smart parking, to provide data-driven services that improve the quality of life for citizens and reduce environmental impact.

1.2 Proposed Solution

The Smart Garage Monitor is a fully automated, cloud-connected IoT system designed to detect the occupancy status of individual parking spots in real time. The system uses ultrasonic distance sensors deployed at each parking spot to determine whether a vehicle is present. This data is processed by a Raspberry Pi gateway, which controls local LED indicators and simultaneously publishes occupancy status to AWS IoT Core via the MQTT protocol over a secure TLS connection.

A serverless web dashboard hosted on Amazon S3 enables any user to monitor all parking spots in real time from a browser, using WebSocket connections authenticated through Amazon Cognito. Email alerts are automatically dispatched via AWS SNS whenever a parking spot changes state (occupied or available), ensuring that both administrators and users are kept informed without manual intervention.

1.3 Objectives

- Detect the real-time occupancy of parking spots using ultrasonic sensors.
- Provide instant local visual feedback to approaching drivers via LED indicators.
- Transmit occupancy data securely to AWS IoT Core using MQTT over TLS.
- Display live parking status on a web dashboard accessible from any device.
- Trigger automated email notifications on state changes using AWS SNS.
- Automate cloud infrastructure provisioning using Terraform (Infrastructure as Code).

1.4 Scope

This project focuses on monitoring two parking spots within a controlled garage environment, serving as a proof-of-concept scalable to larger deployments. The system encompasses the full IoT stack: device-layer sensors and actuators, a Raspberry Pi communication gateway, a secure cloud backend on AWS, and a client-facing web dashboard. The system does not include vehicle identification, license plate recognition, or payment processing.

1.5 Benefits for Al-Alamein Smart City

- Reduces time spent searching for parking, lowering traffic congestion and emissions.
- Provides city administrators with real-time occupancy data for informed management decisions.
- Scales easily: each Raspberry Pi gateway can handle multiple sensors and spots.
- Fully cloud-hosted: no on-premise server infrastructure required.
- Aligns directly with the Al-Alamein smart parking service pillar.

2. Component Selection & Hardware Justification

The Raspberry Pi 4 was selected as the central gateway because it satisfies the project requirement explicitly, and because it provides a full Linux environment suitable for running Python scripts with secure TLS MQTT connections, direct GPIO control, and Terraform-driven cloud configuration. Microcontrollers such as ESP32 or NodeMCU were excluded as they cannot natively establish TLS 1.2 connections to AWS IoT Core without additional hardware.

The HC-SR04 ultrasonic sensor was chosen for its reliability, low cost, and the simplicity of its two-wire digital interface (TRIG/ECHO). It measures distance using acoustic pulse timing, making it ideal for detecting the presence of a vehicle within a defined threshold distance. A critical hardware consideration is the voltage compatibility: the HC-SR04 ECHO pin outputs 5V signals, while the Raspberry Pi GPIO pins operate at 3.3V. A voltage divider circuit consisting of a 1 kΩ and 2 kΩ resistor pair is therefore mandatory on the ECHO line to prevent damage to the GPIO pins.

Red and green LEDs provide immediate, low-cost visual feedback to drivers without requiring connectivity. Each LED is protected by a 330 Ω current-limiting resistor to prevent damage from the GPIO output voltage.

Logic Table

Component	Condition	Action
Ultrasonic Sensor 1 & 2	Distance < 50 cm	Status = Occupied
Red LEDs (Spot 1 & 2)	Status = Occupied	LED ON
Green LEDs (Spot 1 & 2)	Status = Empty	LED ON
AWS SNS	Status Change Detected	Send Email Notification

Sensors & Actuators List

Device	Type	Qty	Function
HC-SR04	Sensor	2	Measure distance to detect vehicle presence
Red LED	Actuator	2	Visual indicator: spot is occupied
Green LED	Actuator	2	Visual indicator: spot is empty
330 Ohm Resistor	Protection	4	Current limiting for LEDs
1kOhm + 2kOhm Resistors	Protection	2 sets	Voltage divider: 5V ECHO to 3.3V safe
Raspberry Pi 4	Gateway	1	Central gateway: GPIO + MQTT + TLS

3. System Inputs & Outputs

Type	Name	Hardware / Service	Description
INPUT	Distance — Car 1 & 2	HC-SR04 #1 & #2	Distance in cm to objects in both spots
OUTPUT	Status LEDs	4x LEDs + resistors	Local visual feedback for approaching drivers
OUTPUT	MQTT Shadow Update	AWS IoT Core	JSON payload with occupancy status and distance
OUTPUT	Email Notification	AWS SNS	Cloud alert triggered on any status change
OUTPUT	Web Dashboard	Amazon S3 + Cognito	Real-time occupancy visible in any browser

4. Hardware Wiring & Circuit Design

4.1 GPIO Pin Assignment (BCM Mode)

GPIO (BCM)	Pin Function	Connected To	Notes
GPIO 17	OUTPUT	HC-SR04 #1 TRIG	Digital pulse to trigger sensor
GPIO 27	INPUT	HC-SR04 #1 ECHO (via divider)	3.3V safe after voltage divider
GPIO 24	OUTPUT	HC-SR04 #2 TRIG	Digital pulse to trigger sensor
GPIO 25	INPUT	HC-SR04 #2 ECHO (via divider)	3.3V safe after voltage divider
GPIO 22	OUTPUT	Red LED — Spot 1	Via 330 Ohm resistor
GPIO 23	OUTPUT	Green LED — Spot 1	Via 330 Ohm resistor
GPIO 5	OUTPUT	Red LED — Spot 2	Via 330 Ohm resistor
GPIO 6	OUTPUT	Green LED — Spot 2	Via 330 Ohm resistor
5V Pin	POWER	HC-SR04 VCC (both)	Both sensors powered from 5V rail
GND Pin	GROUND	All components	Common ground for sensors and LEDs

4.2 Voltage Divider — ECHO Pin Protection

CRITICAL: The HC-SR04 ECHO pin outputs 5V logic, while Raspberry Pi GPIO inputs are rated for 3.3V maximum. Connecting ECHO directly to the GPIO will permanently damage the Raspberry Pi. A resistor voltage divider is required on each ECHO line.

Formula: V_out = V_in x R2 / (R1 + R2) = 5V x 2000 / (1000 + 2000) = 3.33V (within safe range for 3.3V GPIO).

- R1 = 1 kOhm (connected between ECHO pin and GPIO input pin).
- R2 = 2 kOhm (connected between the junction and GND).
- Junction (middle point) connects to the Raspberry Pi GPIO input.

4.3 LED Resistor Calculation

A 330 Ohm current-limiting resistor is placed in series with each LED. With a 3.3V GPIO output and a typical LED forward voltage of 2V, the current is: I = (3.3V - 2V) / 330 Ohm = approximately 4 mA, which is within safe GPIO current limits (max 16 mA per pin).

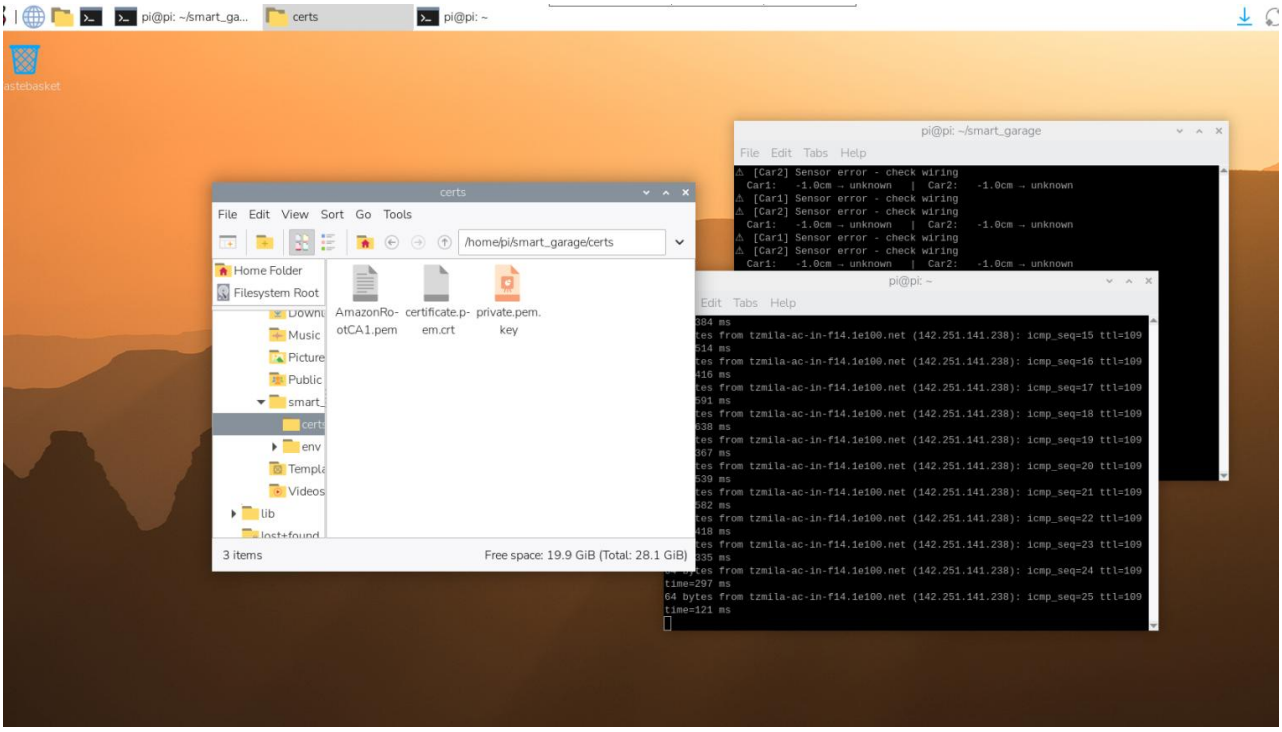
5. Project Directory Structure

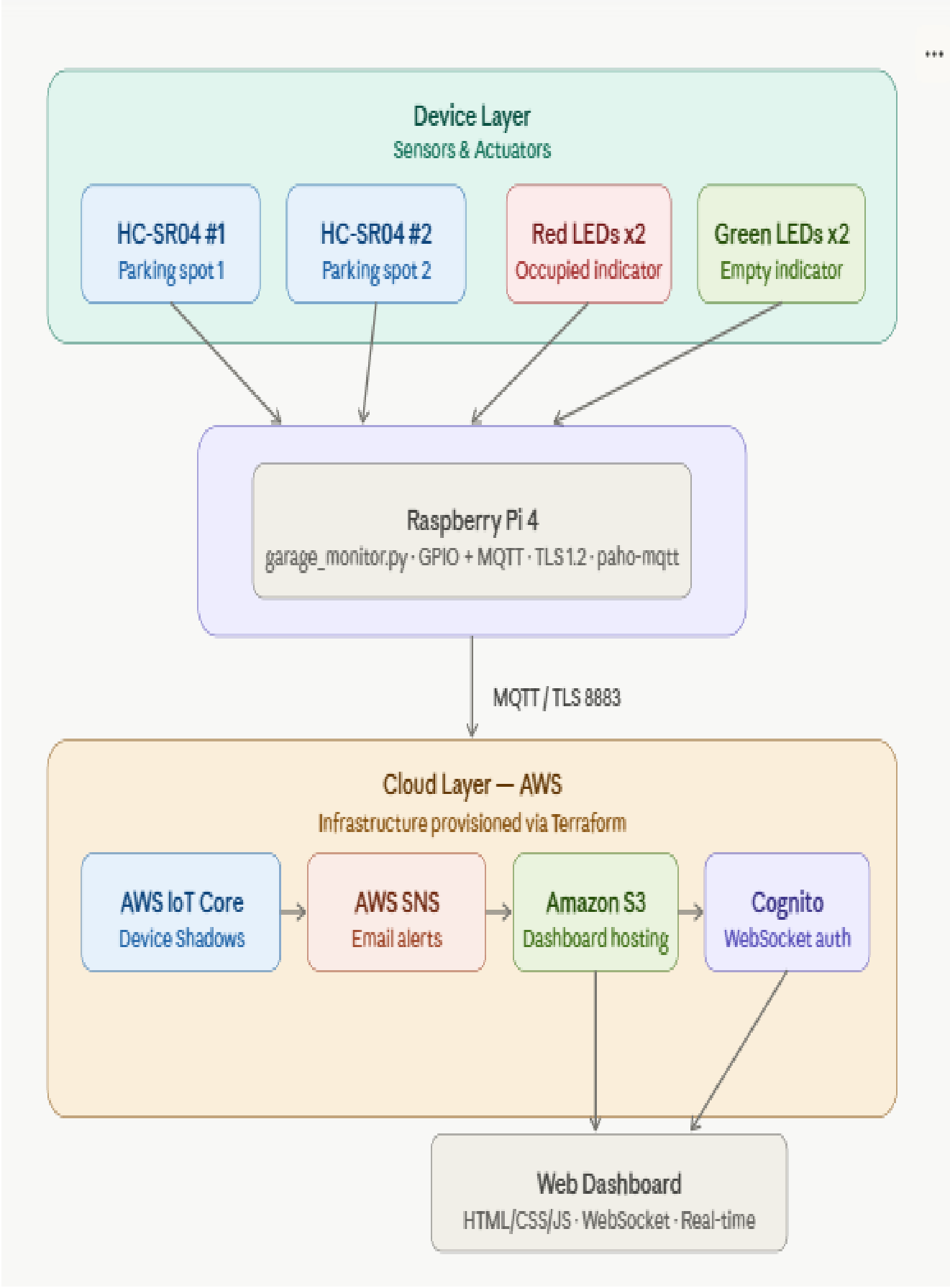
The project follows a clean separation of concerns across four directories:

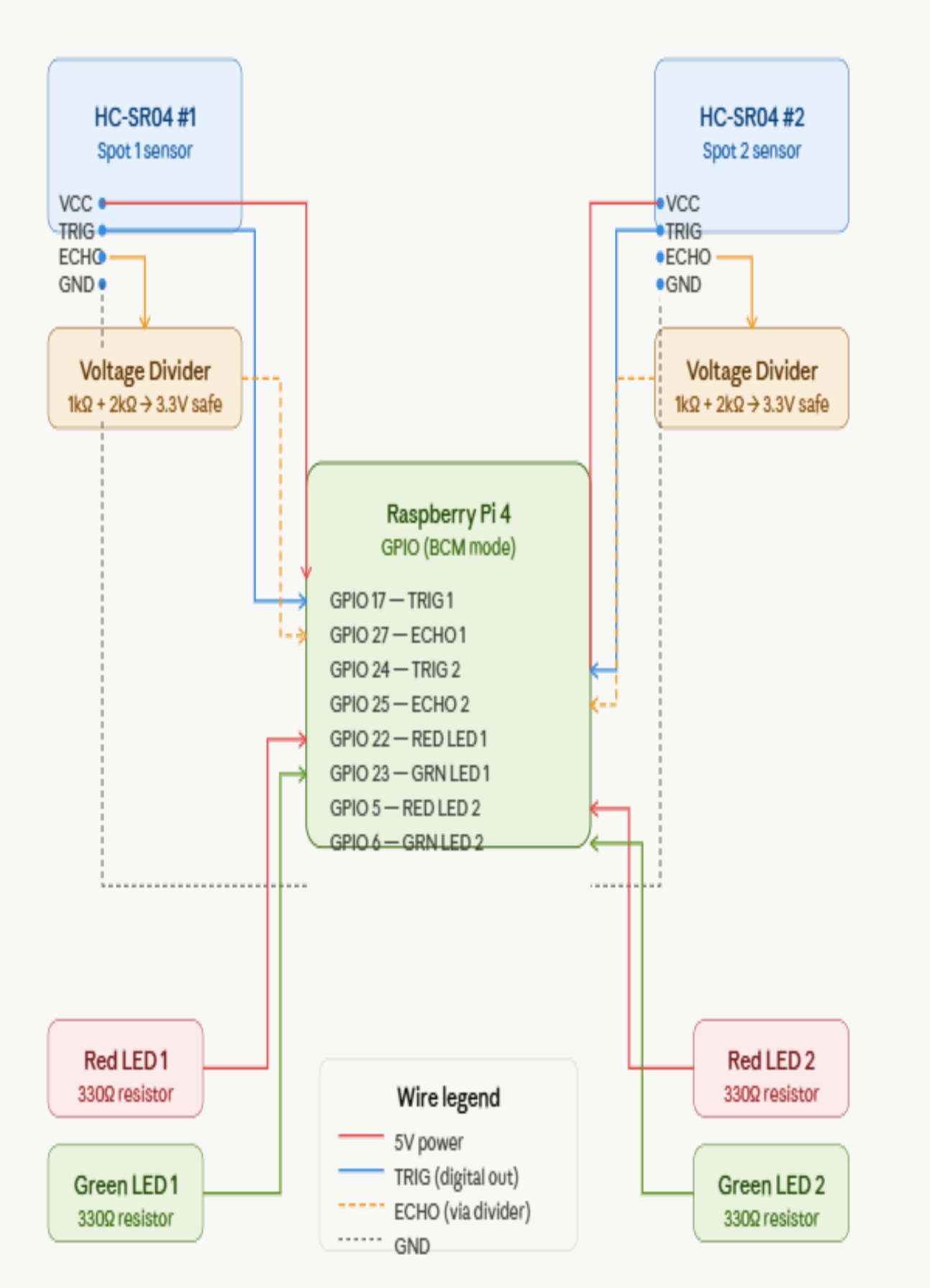
- terraform/ — Infrastructure as Code (IaC) files for provisioning all AWS resources including IoT Core things and certificates, SNS topics, IoT Rules, S3 bucket, and Cognito Identity Pool.
- raspberry-pi/ — Python gateway firmware (garage_monitor.py) and auto-generated TLS certificates.
- website/ — Frontend serverless dashboard: index.html, style.css, app.js.
- scripts/ — Automation shell script (deploy.sh) for one-click infrastructure and deployment.

Project Directory Structure

```
smart-garage-2cars/
├── terraform/                # Infrastructure as Code (AWS Setup)
│   ├── main.tf
│   ├── variables.tf
│   └── outputs.tf
├── raspberry-pi/             # Hardware Gateway Firmware
│   ├── env/                  # Python Virtual Environment
│   ├── garage_monitor.py     # Main Python script
│   └── certs/                # Auto-generated TLS certificates
│       ├── certificate.pem.crt
│       ├── private.pem.key
│       └── AmazonRootCA1.pem
├── website/                  # Frontend Serverless Dashboard
│   ├── index.html
│   ├── style.css
│   └── app.js
└── scripts/                  # Automation
    └── deploy.sh             # One-click deployment script
```


pi@pi: ~/smart_ga...
certs
pi@pi: ~
pi@pi: ~/smart_garage
File Edit Tabs Help
[Car2] Sensor error - check wiring
Car1: -1.0cm → unknown | Car2: -1.0cm → unknown
certs
File Edit View Sort Go Tools
/home/pi/smart_garage/certs
Home Folder
Filesystem Root
Music
Picture
Public
env
Videos
lib
AmazonRo-otCA1.pem
certificate.p-em.crt
private.pem.key
3 items
Free space: 19.9 GiB (Total: 28.1 GiB)
pi@pi: ~
Edit Tabs Help
tzmila-ac-in-f14.1e100.net (142.251.141.238): icmp_seq=15 ttl=109
tzmila-ac-in-f14.1e100.net (142.251.141.238): icmp_seq=16 ttl=109
tzmila-ac-in-f14.1e100.net (142.251.141.238): icmp_seq=17 ttl=109
tzmila-ac-in-f14.1e100.net (142.251.141.238): icmp_seq=18 ttl=109
tzmila-ac-in-f14.1e100.net (142.251.141.238): icmp_seq=19 ttl=109
tzmila-ac-in-f14.1e100.net (142.251.141.238): icmp_seq=20 ttl=109
tzmila-ac-in-f14.1e100.net (142.251.141.238): icmp_seq=21 ttl=109
tzmila-ac-in-f14.1e100.net (142.251.141.238): icmp_seq=22 ttl=109
tzmila-ac-in-f14.1e100.net (142.251.141.238): icmp_seq=23 ttl=109
tzmila-ac-in-f14.1e100.net (142.251.141.238): icmp_seq=24 ttl=109
time=297 ms
64 bytes from tzmila-ac-in-f14.1e100.net (142.251.141.238): icmp_seq=25 ttl=109
time=121 ms


Device Layer
Sensors & Actuators
HC-SR04 #1
Parking spot 1
HC-SR04 #2
Parking spot 2
Red LEDs x2
Occupied indicator
Green LEDs x2
Empty indicator
Raspberry Pi 4
garage_monitor.py · GPIO + MQTT · TLS 1.2 · paho-mqtt
MQTT / TLS 8883
Cloud Layer — AWS
Infrastructure provisioned via Terraform
AWS IoT Core
Device Shadows
AWS SNS
Email alerts
Amazon S3
Dashboard hosting
Cognito
WebSocket auth
Web Dashboard
HTML/CSS/JS · WebSocket · Real-time


HC-SR04 #1
Spot 1 sensor
VCC
TRIG
ECHO
GND
HC-SR04 #2
Spot 2 sensor
VCC
TRIG
ECHO
GND
Voltage Divider
1kΩ + 2kΩ → 3.3V safe
Voltage Divider
1kΩ + 2kΩ → 3.3V safe
Raspberry Pi 4
GPIO (BCM mode)
GPIO 17 — TRIG 1
GPIO 27 — ECHO 1
GPIO 24 — TRIG 2
GPIO 25 — ECHO 2
GPIO 22 — RED LED 1
GPIO 23 — GRN LED 1
GPIO 5 — RED LED 2
GPIO 6 — GRN LED 2
Red LED 1
330Ω resistor
Green LED 1
330Ω resistor
Red LED 2
330Ω resistor
Green LED 2
330Ω resistor
Wire legend
5V power
TRIG (digital out)
ECHO (via divider)
GND

6. Implementation & Integration

Phase 1 — AWS Infrastructure (Terraform)

All AWS cloud resources are provisioned declaratively using Terraform, ensuring reproducibility and eliminating manual configuration. Key resources include: two AWS IoT Things (garage-car1, garage-car2) with auto-generated X.509 certificates; an IoT Topic Rule (GarageStateChange) with an SQL filter that triggers only when a parking spot status changes, publishing to SNS; an S3 bucket configured for static website hosting; and a Cognito Identity Pool granting the browser application unauthenticated WebSocket access to IoT Core.

This sets up IoT Core, Cognito, S3, and SNS to handle message interchange and data hosting securely.

`terraform/main.tf` (Key Snippets)

Terraform

```
# IoT Things & Certs for 2 Cars
resource "aws_iot_thing" "car1" { name = "garage-car1" }
resource "aws_iot_thing" "car2" { name = "garage-car2" }

# SNS & IoT Rule (Email Alerts for any car)
resource "aws_iot_topic_rule" "alert_rule" {
  name        = "GarageStateChange"
  sql         = "SELECT current.state.reported.status as status FROM
'$aws/things/+/shadow/update/documents' WHERE current.state.reported.status
<> previous.state.reported.status"
  sql_version = "2016-03-23"
  enabled     = true
  sns { target_arn = aws_sns_topic.alerts.arn; role_arn =
aws_iam_role.iot_sns.arn; message_format = "RAW" }
}

# S3 Website Hosting
resource "aws_s3_bucket" "web" { bucket = "${var.website_bucket_name}-
${random_id.suff.hex}" }
```

Phase 2 — Raspberry Pi Gateway Firmware

The Python script garage_monitor.py runs continuously on the Raspberry Pi. It initialises two HC-SR04 sensors and four LEDs via RPi.GPIO. Every 2 seconds, it fires a 10-microsecond ultrasonic pulse on each TRIG pin and measures the duration of the ECHO pulse to calculate distance (distance = pulse_duration x 34300 / 2). If the distance is less than 50 cm, the spot is marked occupied; otherwise, it is marked empty.

When a status change is detected, the script publishes a JSON payload to the AWS IoT Device Shadow topic ($aws/things/{thing_name}/shadow/update) via MQTT over TLS port 8883, using the paho-mqtt library authenticated with device certificates generated by Terraform. LED control is updated accordingly on every sensor reading cycle.

An important note on the REPLACE_ME values in app.js: these placeholder values are automatically injected by the deploy.sh automation script at deployment time. The script extracts the live Terraform output values (IoT endpoint, Cognito Pool ID) and writes them into a config.js file before uploading the dashboard to S3. They are not forgotten credentials — they are intentionally left as placeholders for the automation pipeline.

This Python script runs on the Raspberry Pi, acting as the gateway. It implements MQTT for interchanging messages between the physical sensors and AWS IoT Core. *Note: A voltage divider (resistors) is mandatory on the ECHO pins to protect the Raspberry Pi's 3.3V logic from the 5V sensor output.*

raspberry-pi/garage_monitor.py

Python

```
import time
import json
import RPi.GPIO as GPIO
import paho.mqtt.client as mqtt
import ssl

AWS_ENDPOINT = "YOUR_AWS_IOT_ENDPOINT"
THING_CAR1 = "garage-car1"
THING_CAR2 = "garage-car2"

# GPIO Pinout
CAR1_TRIG, CAR1_ECHO, CAR1_RED, CAR1_GREEN = 17, 27, 22, 23
CAR2_TRIG, CAR2_ECHO, CAR2_RED, CAR2_GREEN = 24, 25, 5, 6

GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
# (GPIO Setup for all pins goes here: TRIG/LEDs as OUT, ECHO as IN)

def read_distance(trig_pin, echo_pin):
    GPIO.output(trig_pin, GPIO.HIGH)
    time.sleep(0.00001)
    GPIO.output(trig_pin, GPIO.LOW)

    pulse_start = time.time()
    while GPIO.input(echo_pin) == 0: pulse_start = time.time()
    while GPIO.input(echo_pin) == 1: pulse_end = time.time()

    return round((pulse_end - pulse_start) * 34300 / 2, 2)
```

```
def make_mqtt_client(client_id, cert_path, key_path):
    client = mqtt.Client(client_id=client_id)
    client.tls_set(ca_certs="certs/AmazonRootCA1.pem", certfile=cert_path,
keyfile=key_path, tls_version=ssl.PROTOCOL_TLSv1_2)
    return client

def publish_shadow(client, thing_name, status, distance):
    payload = json.dumps({"state": {"reported": {"status": status,
"distance": distance}}})
    client.publish(f"$aws/things/{thing_name}/shadow/update", payload, qos=1)

def main():
    client1 = make_mqtt_client("rpi-car1", "certs/certificate.pem.crt",
                               "certs/private.pem.key")
    client2 = make_mqtt_client("rpi-car2", "certs/certificate.pem.crt",
                               "certs/private.pem.key")
    client1.connect(AWS_ENDPOINT, 8883)
    client2.connect(AWS_ENDPOINT, 8883)
    client1.loop_start(); client2.loop_start()

    status1 = status2 = "unknown"

    try:
        while True:
            d1 = read_distance(CAR1_TRIG, CAR1_ECHO)
            d2 = read_distance(CAR2_TRIG, CAR2_ECHO)

            new_s1 = "occupied" if d1 < 50 else "empty"
            new_s2 = "occupied" if d2 < 50 else "empty"

            # (LED Control Logic here based on new_s1 and new_s2)

            if new_s1 != status1:
                status1 = new_s1
                publish_shadow(client1, THING_CAR1, status1, d1)

            if new_s2 != status2:
                status2 = new_s2
                publish_shadow(client2, THING_CAR2, status2, d2)

            time.sleep(2)
    except KeyboardInterrupt:
        GPIO.cleanup()

if __name__ == "__main__": main()
```

Phase 3 — Frontend Web Dashboard

The web dashboard is a static, serverless single-page application hosted on Amazon S3. It uses the AWS IoT Device SDK for JavaScript (WebSocket mode) and authenticates via Amazon Cognito Federated Identities. On connection, the browser subscribes to the Device Shadow update/documents topics for both parking spots. Incoming MQTT messages are parsed and the corresponding HTML card is updated in real time with the current status (OCCUPIED / EMPTY) and distance reading.

A user-friendly web interface hosted on Amazon S3 that interacts directly with the IoT system via WebSockets and Cognito.

website/index.html

HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>Smart Garage Monitor</title>
    <link rel="stylesheet" href="style.css">
    <script src="https://sdk.amazonaws.com/js/aws-sdk-
2.1000.0.min.js"></script>
    <script src="https://unpkg.com/aws-iot-device-sdk/dist/aws-iot-device-
sdk-browser.min.js"></script>
</head>
<body>
    <div class="cards-wrapper">
        <div class="card">
            <h2>Spot 1</h2>
            <div id="status-1" class="unknown">--</div>
            <p>Distance: <span id="dist-1">0</span> cm</p>
        </div>
        <div class="card">
            <h2>Spot 2</h2>
            <div id="status-2" class="unknown">--</div>
            <p>Distance: <span id="dist-2">0</span> cm</p>
        </div>
    </div>
    <script src="app.js"></script>
</body>
</html>
```

website/app.js

JavaScript

```
const CONFIG = { poolId: 'REPLACE_ME', host: 'REPLACE_ME', region: 'us-east-
1' };

AWS.config.region = CONFIG.region;
AWS.config.credentials = new AWS.CognitoIdentityCredentials({ IdentityPoolId:
CONFIG.poolId });

const client = awsIot.device({
    region: CONFIG.region, protocol: 'wss', host: CONFIG.host,
```

```
    clientId: 'web' + Math.floor((Math.random() * 100000) + 1),
    accessKeyId: AWS.config.credentials.accessKeyId,
    secretKey: AWS.config.credentials.secretAccessKey,
    sessionToken: AWS.config.credentials.sessionToken
});

client.on('connect', () => {
    client.subscribe(`$aws/things/garage-car1/shadow/update/documents`);
    client.subscribe(`$aws/things/garage-car2/shadow/update/documents`);
});

client.on('message', (topic, payload) => {
    const data = JSON.parse(payload.toString());
    const state = data.current.state.reported;

    if (topic.includes('car1')) {
        document.getElementById('status-1').innerText =
state.status.toUpperCase();
        document.getElementById('status-1').className = state.status;
        document.getElementById('dist-1').innerText = state.distance;
    } else if (topic.includes('car2')) {
        document.getElementById('status-2').innerText =
state.status.toUpperCase();
        document.getElementById('status-2').className = state.status;
        document.getElementById('dist-2').innerText = state.distance;
    }
});
```

Phase 4 — Deployment Automation

The deploy.sh script orchestrates the full deployment pipeline: it runs terraform apply to provision all cloud resources, extracts the output values (IoT endpoint, Cognito Pool ID, S3 bucket name, device certificates), injects the configuration into the JavaScript frontend, uploads the dashboard files to the S3 bucket, and prints the live website URL. This enables a full end-to-end deployment from a single command.

This script ensures seamless integration of software and cloud components by automating infrastructure provisioning and syncing frontend files.

scripts/deploy.sh

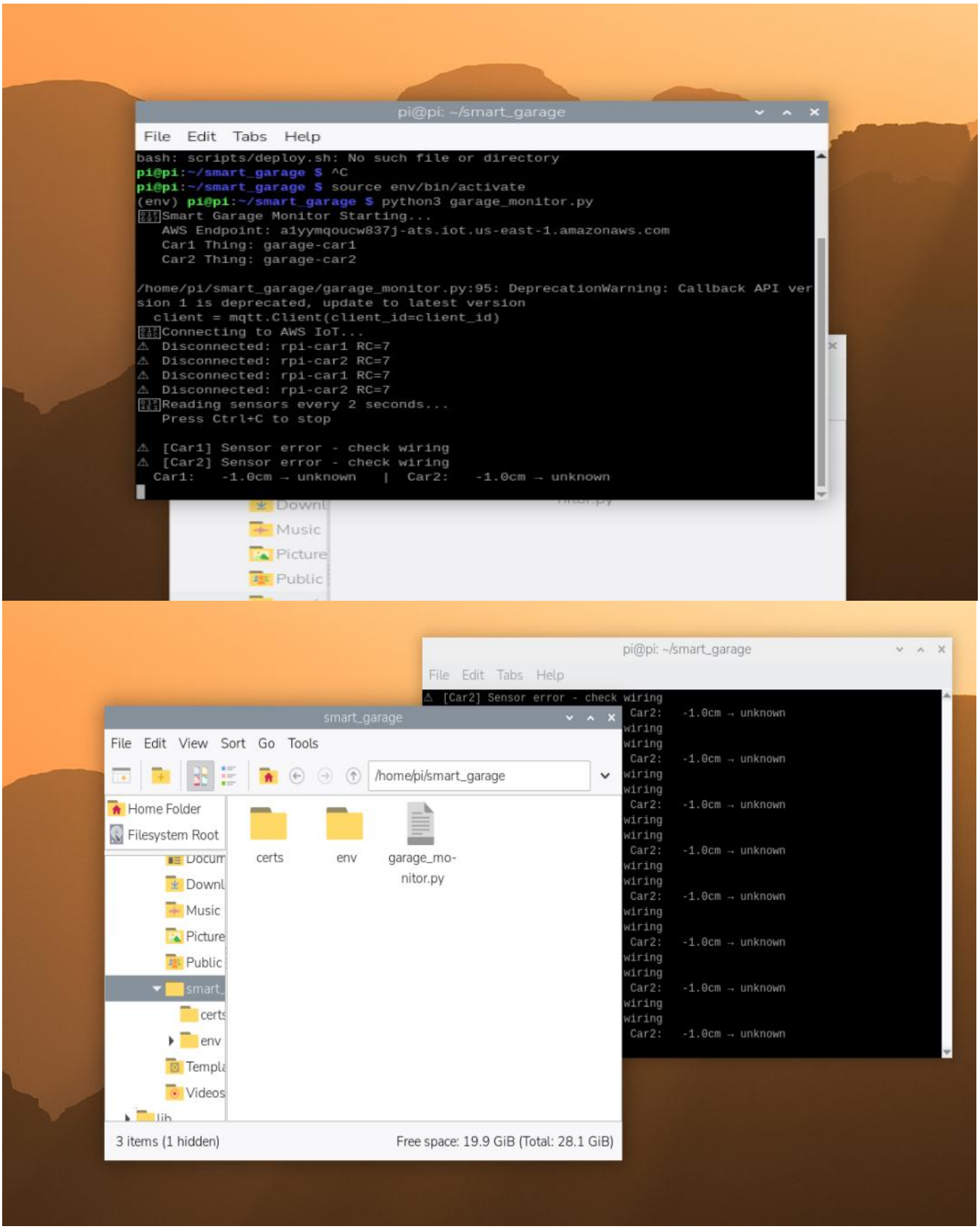
Bash

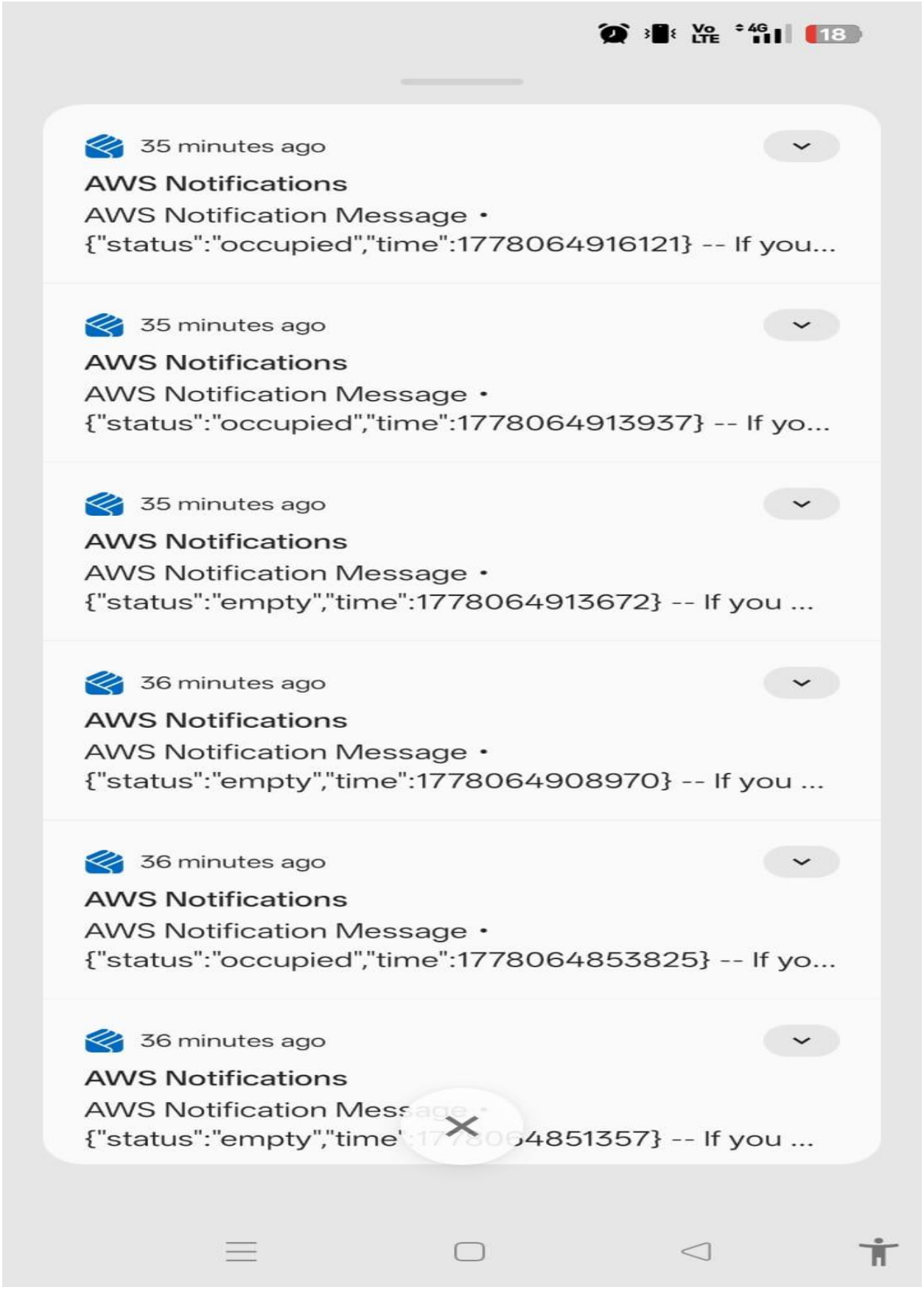
```
#!/bin/bash
set -e
echo "1. Applying Terraform..."
cd terraform
terraform init
terraform apply -auto-approve
output=$(terraform output -json)
cd ..

IOT_ENDPOINT=$(echo $output | jq -r .iot_endpoint.value)
POOL_ID=$(echo $output | jq -r .cognito_pool_id.value)
BUCKET=$(echo $output | jq -r .bucket_name.value)

echo "2. Generating Certificates for Raspberry Pi..."
mkdir -p raspberry-pi/certs
echo $output | jq -r .car1_cert_pem.value > raspberry-
pi/certs/certificate.pem.crt
echo $output | jq -r .car1_key_pem.value > raspberry-pi/certs/private.pem.key
curl -s -o raspberry-pi/certs/AmazonRootCA1.pem
https://www.amazontrust.com/repository/AmazonRootCA1.pem

echo "3. Injecting JS Config..."
echo "const CONFIG = { poolId: '$POOL_ID', host: '$IOT_ENDPOINT', region:
'us-east-1' };" > website/config.js

echo "4. Uploading Dashboard to S3..."
aws s3 sync website/ s3://$BUCKET/

echo "✅ Done! Website URL: http://$BUCKET.s3-website-us-east-1.amazonaws.com"
```

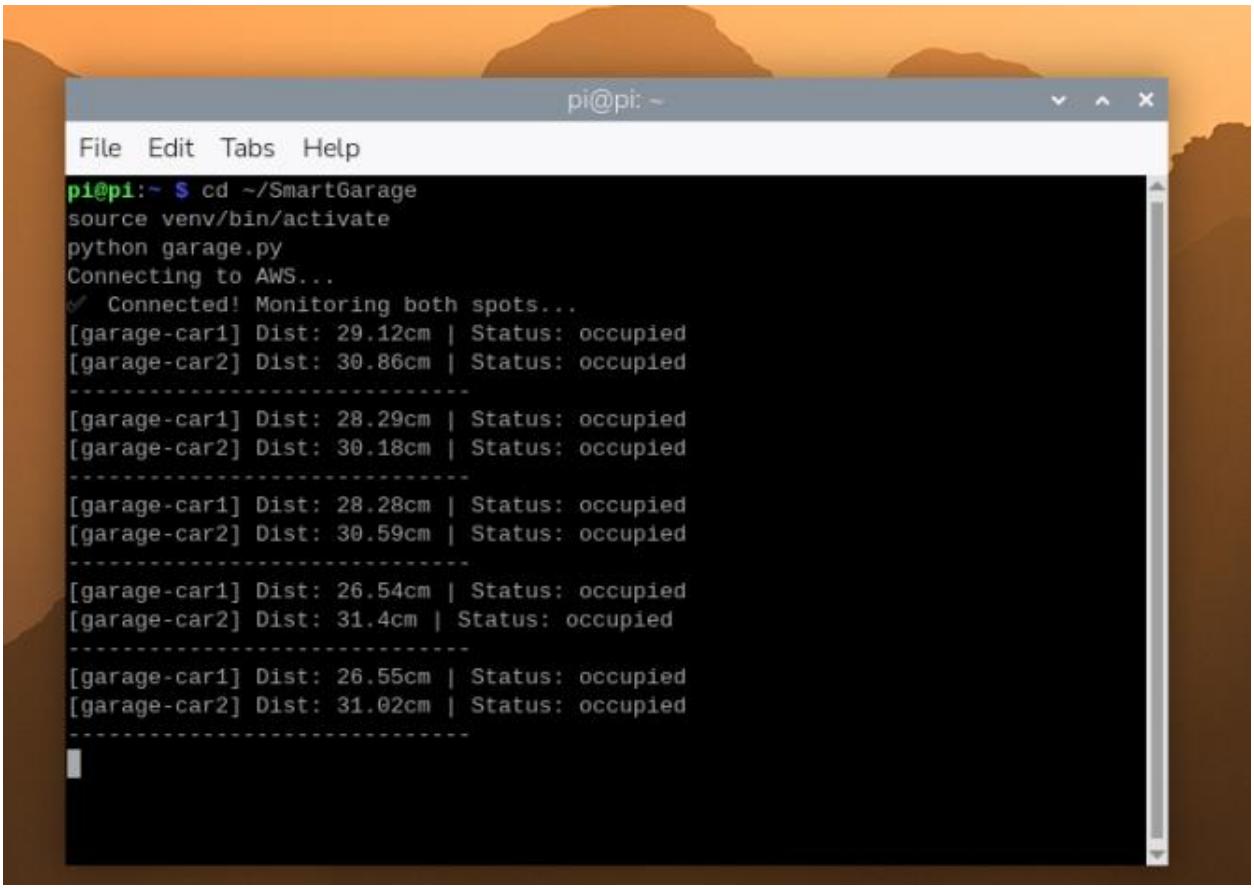
---

7. Test Strategy

#	Test Name	Procedure	Expected Result
1	Hardware Sensor Test	Place hand < 50 cm from HC-SR04 #1	Red LED 1 turns ON; Green LED 1 turns OFF
2	MQTT Connection Test	Run garage_monitor.py and observe terminal	Terminal prints confirmation of successful AWS IoT Core connection
3	Dashboard UI Test	Move object in front of sensor #2 and observe browser	Web dashboard updates Spot 2 to OCCUPIED within 3 seconds
4	Integration & Alert Test	Block both sensors simultaneously	Dashboard shows both spots OCCUPIED; email alert received from AWS SNS
5	Empty State Test	Remove objects from both sensors	Both spots return to EMPTY on dashboard; Green LEDs turn ON; email alert sent
6	Voltage Divider Safety Test	Measure ECHO voltage at GPIO input pin with multimeter	Voltage reading at GPIO pin is approximately 3.3V (not 5V)


pi@pi: ~/smart_garage
File Edit Tabs Help
bash: scripts/deploy.sh: No such file or directory
pi@pi:~/smart_garage $ ^C
pi@pi:~/smart_garage $ source env/bin/activate
(env) pi@pi:~/smart_garage $ python3 garage_monitor.py
Smart Garage Monitor Starting...
AWS Endpoint: a1yymqoucw837j-ats.iot.us-east-1.amazonaws.com
Car1 Thing: garage-car1
Car2 Thing: garage-car2
/home/pi/smart_garage/garage_monitor.py:95: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
client = mqtt.Client(client_id=client_id)
Connecting to AWS IoT...
⚠ Disconnected: rpi-car1 RC=7
⚠ Disconnected: rpi-car2 RC=7
⚠ Disconnected: rpi-car1 RC=7
⚠ Disconnected: rpi-car2 RC=7
Reading sensors every 2 seconds...
Press Ctrl+C to stop
⚠ [Car1] Sensor error - check wiring
⚠ [Car2] Sensor error - check wiring
Car1: -1.0cm → unknown | Car2: -1.0cm → unknown
Music
Picture
Public
pi@pi: ~/smart_garage
File Edit Tabs Help
⚠ [Car2] Sensor error - check wiring
Car2: -1.0cm → unknown
smart_garage
File Edit View Sort Go Tools
/home/pi/smart_garage
Home Folder
Filesystem Root
certs
env
garage_monitor.py
Music
Picture
Public
Videos
3 items (1 hidden)
Free space: 19.9 GiB (Total: 28.1 GiB)


35 minutes ago
AWS Notifications
AWS Notification Message •
{"status":"occupied","time":1778064916121} -- If you...
35 minutes ago
AWS Notifications
AWS Notification Message •
{"status":"occupied","time":1778064913937} -- If yo...
35 minutes ago
AWS Notifications
AWS Notification Message •
{"status":"empty","time":1778064913672} -- If you ...
36 minutes ago
AWS Notifications
AWS Notification Message •
{"status":"empty","time":1778064908970} -- If you ...
36 minutes ago
AWS Notifications
AWS Notification Message •
{"status":"occupied","time":1778064853825} -- If yo...
36 minutes ago
AWS Notifications
AWS Notification Mess
{"status":"empty","time
4851357} -- If you ...



Running:

1. **تدخل فولدر المشروع:** `cd ~/smart_garage`
2. **تشغل بيئة البايثون (عشان المكتبات):** `source env/bin/activate`
3. **تشغل الكود بتاعك:** `python3 garage_monitor.py`

8. Conclusion

The Smart Garage Monitor successfully demonstrates the practical application of IoT technology in a smart city context. The system integrates hardware sensors, a Raspberry Pi gateway, MQTT-based cloud communication, and a real-time web interface into a cohesive, deployable solution that addresses the urban challenge of inadequate parking management identified in the Al-Alamein Smart City framework.

The use of AWS cloud services (IoT Core, SNS, S3, Cognito) ensures scalability, security through TLS-encrypted MQTT connections, and zero on-premise server infrastructure. The Terraform-based Infrastructure as Code approach guarantees reproducible deployments and aligns with professional DevOps practices. The project meets all stated requirements: Raspberry Pi as the communication gateway, MQTT for device-to-cloud messaging, cloud services for data processing and storage, and a user-friendly web interface for real-time monitoring.

Team members

Name	Student ID
Yusuf Yasser Said	22100809
Ahmed Hazem	22101014
Mohammed Essam	21100810